

Proxylab(2)

实验环境

1. VScode
2. Ubuntu 20.04.6 LTS

实验要求

代码逻辑清晰，必要处有注释实验报告需要有详细的分析过程和说明

实验内容

1. 在 Part1 的基础上分别实现带有多线程并发处理功能、利用LRU策略实现缓存功能的网络代理服务器，[并通过运行driver.sh](#) 测试脚本检验功能实现。要求代码关键处有注释，测试得分情况有截图

```
#include <stdio.h>
#include "csapp.h"
/* 202212063020 */
#define MAX_CACHE_SIZE 1049000
#define MAX_OBJECT_SIZE 102400
static const char *user_agent_hdr = "User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:10.0.3) Gecko/20120305
Firefox/10.0.3\r\n";
void doit(int clientfd);
int parse_uri(char *uri, char *hostname, char *path, char *port, char *request_head);
void read_requesthdrs(rio_t *rp, int fd);
void return_content(int serverfd, int clientfd);
void *thread(void *vargp);
int main(int argc, char **argv)
{
    int listenfd;
    int *conncfd; /* 使用指针, 避免竞争 */
    char hostname[MAXLINE], port[MAXLINE];
    socklen_t clientlen;
    struct sockaddr_storage clientaddr;
    pthread_t tid;
    if(argc != 2)
    {
        fprintf(stderr, "usage: %s <port>\n", argv[0]);
        exit(1);
    }
    /* 代理创建一个监听描述符, 准备好接收连接请求 */
    listenfd = Open_listenfd(argv[1]);
    while(1)
    {
        clientlen = sizeof(clientaddr);
        conncfd = Malloc(sizeof(int));
        /* 等待来着客户端的连接请求到侦听描述符 listenfd,
        然后在 addr 中填写客户端的套接字地址, 并返回一个已连接描述符
        */
        *conncfd = Accept(listenfd, (SA *)&clientaddr, &clientlen);
        /* 将套接字地址结构 clientaddr 转化成对应的主机和服务名字符串,
        并将他们复制到 hostname 和 port 缓冲区
        */
        Getnameinfo((SA *)&clientaddr, clientlen, hostname, MAXLINE, port, MAXLINE, 0);
        printf("Accepted connection from (%s, %s)\n", hostname, port);
        /* 调用 pthread_create 函数来创建其他线程 */
        Pthread_create(&tid, NULL, thread, conncfd);
    }
}
void *thread(void *vargp)
{
    -- INSERT --
}
```

```
void *thread(void *vargp)
{
    int connfd = *((int *)vargp);
    Pthread_detach(pthread_self());
    Free(vargp);
    doit(connfd);
    Close(connfd);
    return NULL;
}
/* 处理客户端 HTTP 事务 */
void doit(int clientfd)
{
    char buf[MAXLINE], method[MAXLINE], uri[MAXLINE], version[MAXLINE];
    char hostname[MAXLINE], path[MAXLINE], port[MAXLINE], request_head[MAXLINE];
    int serverfd;
    rio_t rio;
    /* 读取请求行和请求头 */
    Rio_readinitb(&rio, clientfd);
    Rio_readlineb(&rio, buf, MAXLINE);
    sscanf(buf, "%s %s %s", method, uri, version);
    if(strcasecmp(method, "GET"))
    {
        printf("Not implemented");
        return;
    }
    /* 解析 uri 取 hostname 和 path 和 port。生成 request_head */
    parse_uri(uri, hostname, path, port, request_head);
    /* 建立与服务器的连接 */
    serverfd = Open_clientfd(hostname, port);
    /* 将请求头传递给服务器 */
    Rio_writen(serverfd, request_head, strlen(request_head));
    read_requesthdrs(&rio, serverfd);
    /* 将服务端读取的数据返回给客户端 */
    return_content(serverfd, clientfd);
}
/* 解析 uri 中的 hostname 和 path 和 port。并生成 request_head
* uri 例子: http://www.cmu.edu:8080/hub/index.html
* hostname: www.cmu.edu
* path: /hub/index.html
* port: 8080
*/
int parse_uri(char *uri, char *hostname, char *path, char *port, char *request_head)
{
    sprintf(port, "80"); //默认值
    char *end, *bp;
    char *tail = uri + strlen(uri); //uri 的最后一个字符, 不是'\0'。
-- INSERT --
```

```

char *bg = strstr(uri,"//");
bg = (bg!=NULL ? bg+2 : uri); //取 hostname 的开头。
end = bg;
//取 hostname 的结尾。
while(*end != '/' && *end != ':') end++;
strncpy(hostname,bg,end-bg);
bp = end + 1; //取 port 的开头
if(*end == ':')
{
    //==':'说明 uri 中有 port
    end++;
    bp = strstr(bg,"/"); //取 port 的结尾
    strncpy(port,end,bp-end);
    end = bp; //取 uri 的开头
}
strncpy(path,end,(int)(tail-end)+1);
/* 请求的开头行: GET /hub/index.html HTTP/1.0。 */
sprintf(request_head,"GET %s HTTP/1.0\r\nHost: %s\r\n",path,hostname);
return 1;
}
/*
* 读取 HTTP 请求头
* Host,User-Agent,Connection 和 Proxy-Connection 用指定的的
* 保留其他的头(header)
*/
void read_requesthdrs(rio_t *rp,int fd)
{
    char buf[MAXLINE];
    sprintf(buf, "%s", user_agent_hdr);
    Rio_writen(fd, buf, strlen(buf));
    sprintf(buf, "Connection: close\r\n");
    Rio_writen(fd, buf, strlen(buf));
    sprintf(buf, "Proxy-Connection: close\r\n");
    Rio_writen(fd, buf, strlen(buf));
    /* 保留其他的头(header) */
    for(Rio_readlineb(rp,buf,MAXLINE);strcmp(buf,"\r\n");Rio_readlineb(rp,buf,MAXLINE))
    {
        if(strncmp("Host",buf,4) == 0 || strncmp("User-Agent",buf,10) == 0
           || strncmp("Connection",buf,10) == 0 || strncmp("Proxy-Connection",buf,16) == 0)
            continue;
        printf("%s",buf);
        Rio_writen(fd,buf,strlen(buf));
    }
    Rio_writen(fd,buf,strlen(buf));
    return;
}
-- INSERT --

```

```
}
strncpy(path,end,(int)(tail-end)+1);
/* 请求的开头行: GET /hub/index.html HTTP/1.0。 */
sprintf(request_head,"GET %s HTTP/1.0\r\nHost: %s\r\n",path,hostname);
return 1;
}
/*
* 读取 HTTP 请求头
* Host,User-Agent,Connection 和 Proxy-Connection 用指定的的
* 保留其他的头(header)
*/
void read_requesthdrs(rio_t *rp,int fd)
{
    char buf[MAXLINE];
    sprintf(buf,"%s", user_agent_hdr);
    Rio_writen(fd, buf, strlen(buf));
    sprintf(buf,"Connection: close\r\n");
    Rio_writen(fd, buf, strlen(buf));
    sprintf(buf,"Proxy-Connection: close\r\n");
    Rio_writen(fd, buf, strlen(buf));
    /* 保留其他的头(header) */
    for(Rio_readlineb(rp,buf,MAXLINE);strcmp(buf,"\r\n");Rio_readlineb(rp,buf,MAXLINE))
    {
        if(strncmp("Host",buf,4) == 0 || strncmp("User-Agent",buf,10) == 0
           || strncmp("Connection",buf,10) == 0 || strncmp("Proxy-Connection",buf,16) == 0)
            continue;
        printf("%s",buf);
        Rio_writen(fd,buf,strlen(buf));
    }
    Rio_writen(fd,buf,strlen(buf));
    return;
}
/*
* 将服务端读取的数据返回给客户端
*/
void return_content(int serverfd, int clientfd)
{
    size_t n;
    char buf[MAXLINE];
    rio_t srrio;
    Rio_readinitb(&srrio,serverfd);
    while((n = Rio_readlineb(&srrio,buf,MAXLINE)) != 0)
    {
        Rio_writen(clientfd,buf,n);
    }
}
-- INSERT --
```

```
*** Basic ***
Starting tiny on 10036
Starting proxy on 9734
1: home.html
  Fetching ./tiny/home.html into ../.proxy using the proxy
  Fetching ./tiny/home.html into ../.noproxy directly from Tiny
  Comparing the two files
  Success: Files are identical.
2: csapp.c
  Fetching ./tiny/csapp.c into ../.proxy using the proxy
  Fetching ./tiny/csapp.c into ../.noproxy directly from Tiny
  Comparing the two files
  Success: Files are identical.
3: tiny.c
  Fetching ./tiny/tiny.c into ../.proxy using the proxy
  Fetching ./tiny/tiny.c into ../.noproxy directly from Tiny
  Comparing the two files
  Success: Files are identical.
4: godzilla.jpg
  Fetching ./tiny/godzilla.jpg into ../.proxy using the proxy
  Fetching ./tiny/godzilla.jpg into ../.noproxy directly from Tiny
  Comparing the two files
  Success: Files are identical.
5: tiny
  Fetching ./tiny/tiny into ../.proxy using the proxy
  Fetching ./tiny/tiny into ../.noproxy directly from Tiny
  Comparing the two files
  Success: Files are identical.
Killing tiny and proxy
basicScore: 40/40
```

2. 利用 Curl 指令实现通过代理对Tiny服务器页面的访问。具体实现方式参考第 15 周实验课 ppt 末尾。也可尝试对其他页面进行访问。要求有curl 指令所在窗口的响应截图。

```
#include <stdio.h>
#include "csapp.h"
/* 202212063020 */
#define MAX_CACHE_SIZE 1049000
#define MAX_OBJECT_SIZE 102400
#define MAX_CACHE 10
static const char *user_agent_hdr = "User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:10.0.3) Gecko/20120305 Firefox/10.0.3\r\n";
void doit(int clientfd);
int parse_uri(char *uri, char *hostname, char *path, char *port, char *request_head);
void read_requesthdrs(rio_t *rp, int fd);
void return_content(int serverfd, int clientfd, char *url);
void *thread(void *vargp);
int maxlrucache();
/*读写者锁的结构体*/
struct RWLOCK_T
{
    sem_t lock; //基本锁
    sem_t writeLock; //写着锁
    int readcnt; //读者个数
};
/*LRU 缓存的结构体*/
struct CACHE
{
    int lruNumber; //引用次数, 根据大小排位。大的表示最近引用过
    char url[MAXLINE]; //通过 url 唯一标识对应 content
    char content[MAX_OBJECT_SIZE];
};
struct CACHE cache[MAX_CACHE]; //缓存, 最多有 MAX_CACHE 个
struct RWLOCK_T* rw; //读写者锁指针
void rwlock_init(); //初始化读写者锁指针
char *readcache(char *url); //读缓存
void writcache(char *buf, char *url); //写缓存
int main(int argc, char **argv)
{
    int listenfd;
    int *connfd; /* 使用指针, 避免竞争 */
    char hostname[MAXLINE], port[MAXLINE];
    socklen_t clientlen;
    struct sockaddr_storage clientaddr;
    pthread_t tid;
    if(argc != 2)
    {
        fprintf(stderr, "usage: %s <port>\n", argv[0]);
        exit(1);
    }
    -- INSERT --
```

```
rw = Malloc(sizeof(struct RWLOCK_T));
rwlock_init();
/* 代理创建一个监听描述符, 准备好接收连接请求 */
listenfd = Open_listenfd(argv[1]);
while(1)
{
    clientlen = sizeof(clientaddr);
    connfd = Malloc(sizeof(int));
    /* 等待来着客户端的连接请求到侦听描述符 listenfd,
    然后在 addr 中填写客户端的套接字地址, 并返回一个已连接描述符
    */
    *connfd = Accept(listenfd, (SA *)&clientaddr, &clientlen);
    /* 将套接字地址结构 clientaddr 转化成对应的主机和服务名字符串,
    并将他们复制到 hostname 和 port 缓冲区
    */
    Getnameinfo((SA *)&clientaddr, clientlen, hostname, MAXLINE, port, MAXLINE, 0);
    printf("Accepted connection from (%s, %s)\n", hostname, port);
    /* 调用 pthread_create 函数来创建其他线程 */
    Pthread_create(&tid, NULL, thread, connfd);
}
}
void *thread(void *vargp)
{
    int connfd = *((int *)vargp);
    Pthread_detach(pthread_self());
    Free(vargp);
    doit(connfd);
    Close(connfd);
    return NULL;
}
/* 处理客户端 HTTP 事务 */
void doit(int clientfd)
{
    char buf[MAXLINE], method[MAXLINE], uri[MAXLINE], version[MAXLINE];
    char hostname[MAXLINE], path[MAXLINE], port[MAXLINE], request_head[MAXLINE];
    int serverfd;
    rio_t rio;
    /* 读取请求行和请求头 */
    Rio_readinitb(&rio, clientfd);
    Rio_readlineb(&rio, buf, MAXLINE);
    sscanf(buf, "%s %s %s", method, uri, version);
    if(strcasecmp(method, "GET"))
    {
        printf("Not implemented");
        return;
    }
}
-- INSERT --
```



```
char *content = readcache(uri);
if(content != NULL)
{
    Rio_writen(clientfd,content,strlen(content));
    free(content);
}
else
{
    /* 解析 uri 取 hostname 和 path 和 port。生成 request_head */
    parse_uri(uri,hostname,path,port,request_head);
    /* 建立与服务器的连接 */
    serverfd = Open_clientfd(hostname,port);
    /* 将请求头传递给服务器 */
    Rio_writen(serverfd,request_head,strlen(request_head));
    read_requesthdrs(&rio,serverfd);
    /* 将服务端读取的数据返回给客户端 */
    return_content(serverfd,clientfd,uri);
}
}
/* 解析 uri 中的 hostname 和 path 和 port。并生成 request_head
* uri 例子: http://www.cmu.edu:8080/hub/index.html
* hostname:www.cmu.edu
* path:/hub/index.html
* port:8080
*/
int parse_uri(char *uri,char *hostname,char *path,char *port,char *request_head)
{
    sprintf(port,"80"); //默认值
    char *end,*bp;
    char *tail = uri+strlen(uri); //uri 的最后一个字符, 不是'\0'。
    char *bg = strstr(uri,"//");
    bg = (bg!=NULL ? bg+2 : uri); //取 hostname 的开头。
    end = bg;
    //取 hostname 的结尾。
    while(*end != '/' && *end != ':') end++;
    strncpy(hostname,bg,end-bg);
    bp = end + 1; //取 port 的开头
    if(*end == ':')
    {
        //==':'说明 uri 中有 port
        end++;
        bp = strstr(bg,"/"); //取 port 的结尾
        strncpy(port,end,bp-end);
        end = bp; //取 uri 的开头
    }
    strncpy(path,end,(int)(tail-end)+1);
-- INSERT --
```

```
/* 请求的开头行: GET /hub/index.html HTTP/1.0. */
sprintf(request_head, "GET %s HTTP/1.0\r\nHost: %s\r\n", path, hostname);
return 1;
}
/*
* 读取 HTTP 请求头
* Host, User-Agent, Connection 和 Proxy-Connection 用指定的
* 保留其他的头(header)
*/
void read_requesthdrs(rio_t *rp, int fd)
{
    char buf[MAXLINE];
    sprintf(buf, "%s", user_agent_hdr);
    Rio_writen(fd, buf, strlen(buf));
    sprintf(buf, "Connection: close\r\n");
    Rio_writen(fd, buf, strlen(buf));
    sprintf(buf, "Proxy-Connection: close\r\n");
    Rio_writen(fd, buf, strlen(buf));
    /* 保留其他的头(header) */
    for(Rio_readlineb(rp, buf, MAXLINE); strcmp(buf, "\r\n"); Rio_readlineb(rp, buf, MAXLINE))
    {
        if(strncmp("Host", buf, 4) == 0 || strncmp("User-Agent", buf, 10) == 0
           || strncmp("Connection", buf, 10) == 0 || strncmp("Proxy-Connection", buf, 16) == 0)
            continue;
        printf("%s", buf);
        Rio_writen(fd, buf, strlen(buf));
    }
    Rio_writen(fd, buf, strlen(buf));
    return;
}
/*
* 将服务端读取的数据返回给客户端
*/
void return_content(int serverfd, int clientfd, char *uri)
{
    size_t n, size = 0;
    char buf[MAXLINE], content[MAX_OBJECT_SIZE];
    rio_t srio;
    Rio_readinitb(&srio, serverfd);
    while((n = Rio_readlineb(&srio, buf, MAXLINE)) != 0)
    {
        Rio_writen(clientfd, buf, n);
        if(n + size <= MAX_OBJECT_SIZE)
        {
            sprintf(content + size, "%s", buf);
            size += n;
        }
    }
}
-- INSERT --
```



```
    }
    else
    {
        size = MAX_OBJECT_SIZE + 1;
    }
}
writecache(content,uri);
}
/*-----cache start-----*/
void rwlock_init()
{
    rw->readcnt = 0;
    sem_init(&rw->lock,0,1);
    sem_init(&rw->writeLock,0,1);
}
void writecache(char *buf,char *url)
{
    sem_wait(&rw->writeLock); //等待获得写者锁
    int index;
    /*看看缓存有没有空位*/
    for(index = 0;index < MAX_CACHE;index++)
    {
        if(cache[index].lruNumber == 0)
        {
            break;
        }
    }
    /*没有空位, 根据 LRU 政策驱逐*/
    if(index == MAX_CACHE)
    {
        int minlru = cache[0].lruNumber;
        /*找到最后访问的缓存*/
```

/*找到最后访问过的缓存*/

```
for(int i = 1;i < MAX_CACHE;i++)  
{  
    if(cache[i].lruNumber < minlru)  
    {  
        minlru = cache[i].lruNumber;  
        index = i;  
    }  
}
```

```
| }  
cache[index].lruNumber = maxlrucache()+1;  
strcpy(cache[index].url,url);  
strcpy(cache[index].content,buf);  
sem_post(&rw->writeLock); //释放锁  
return;
```

```
-- INSERT --
```



```
    cache[index].lruNumber = maxlrucache()+1;
    strcpy(cache[index].url,url);
    strcpy(cache[index].content,buf);
    sem_post(&rw->writeLock); //释放锁
    return;
}
char *readcache(char *url)
{
    sem_wait(&rw->lock); //读者等待并获取锁
    if(rw->readcnt == 1)
        sem_wait(&rw->writeLock); //读者在读，不允许有写者
    rw->readcnt++;
    sem_post(&rw->lock); //释放锁
    char *content = NULL;
    for(int i = 0;i < MAX_CACHE;i++)
    {
        /*找到了对应的缓存*/
        if(strcmp(url,cache[i].url) == 0)
        {
            content = (char *)Malloc(strlen(cache[i].content));
            strcpy(content,cache[i].content);
            int maxlru = maxlrucache(); //获取最大的 lru
            cache[i].lruNumber = maxlru+1; //+1 成最大的 lru
            break;
        }
    }
    sem_wait(&rw->lock); //等待并获取锁
    rw->readcnt--; if(rw->readcnt == 0) //没有读者了，释放写者锁
    sem_post(&rw->writeLock);
    sem_post(&rw->lock); //释放锁
    return content;
}
int maxlrucache()
{
    int i;
    int max=0;
    for(i = 0;i<MAX_CACHE;i++)
    {
        if(cache[i].lruNumber > max)
        {
            max = cache[i].lruNumber;
        }
    }
}
```

```
    }  
  }  
  return max;  
}  
/*-----cache end-----*/  
-- INSERT --
```

*** Concurrency ***

Starting tiny on port 24057

Starting proxy on port 6027

Starting the blocking NOP server on port 1048

Trying to fetch a file from the blocking nop-server

Fetching ./tiny/home.html into ../.noproxy directly from Tiny

Fetching ./tiny/home.html into ../.proxy using the proxy

Checking whether the proxy fetch succeeded

Success: Was able to fetch tiny/home.html from the proxy.

Killing tiny, proxy, and nop-server

concurrencyScore: 15/15

*** Cache ***

Starting tiny on port 33531

Starting proxy on port 17610

Fetching ./tiny/tiny.c into ../.proxy using the proxy

Fetching ./tiny/home.html into ../.proxy using the proxy

Fetching ./tiny/csapp.c into ../.proxy using the proxy

Killing tiny

Fetching a cached copy of ./tiny/home.html into ../.noproxy

Success: Was able to fetch tiny/home.html from the cache.

Killing proxy

cacheScore: 15/15

totalScore: 70/70